Profiling cho AI model

Presentated by Group 3

Our Team

Nguyễn Hữu Nhật Hoàng

Phan Quốc Hùng

Nội Dung

Tổng quan đề tài

Nghiên cứu và phân tích tài liệu

Demo sản phẩm

Tổng quan đề tài

Mục tiêu đề tài

Chất lượng mô hình

Đánh giá Accuracy, Precision, Recall, F1-score hoặc metric phù hợp với từng bài toán.

Hiệu năng thực thi

Đo latency, throughput, thời gian xử lý và thời gian tải model khi inference.

Tài nguyên sử dụng

Theo dõi CPU/GPU utilization, RAM, VRAM và tài nguyên liên quan.

Layer / Operator Profiling

Mở rộng phân tích xuống từng layer/operator để xác định bottleneck và hỗ trợ tối ưu model hoặc môi trường triển khai.

Hướng tiếp cận của đề tài

Người dùng có thể

Tải lên hoặc khai báo model
Chọn dataset kiểm thử
Chọn runtime và phần cứng

Hệ thống thực hiện

Benchmark tự động
Thu thập metric
Xuất báo cáo Model Profile

Chức năng chính

Đánh giá chất lượng • Đo latency/throughput/tài nguyên • Phân tích bottleneck • So sánh model trên nhiều runtime và phần cứng

Đầu vào của hệ thống

Mô hình đã huấn luyện

Đối tượng chính cần profiling, gồm kiến trúc tính toán và trọng số đã học.

File đặc tả

Mô tả cách sử dụng model: tên, phiên bản, framework, input type, input shape...

Dataset đánh giá

Dữ liệu dùng để chạy inference và tính các metric chất lượng.

Benchmark Configuration

Cấu hình môi trường mà người dùng muốn sử dụng để kiểm thử model: runtime, hardware,...

Đầu ra của hệ thống

Thông tin mô hình
Tên model, định dạng, runtime sử dụng.

Môi trường thực thi
CPU/GPU, batch size, precision.

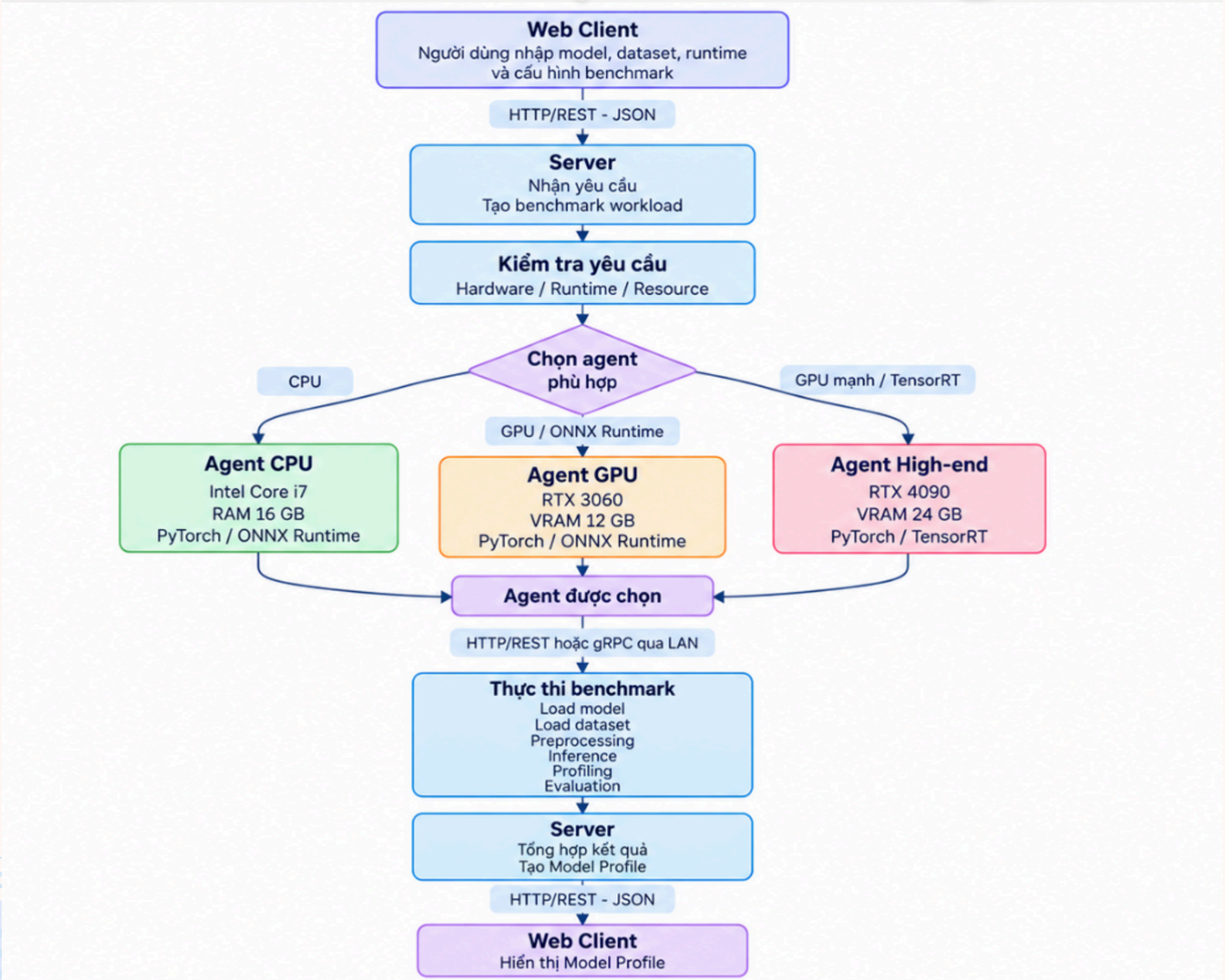
Chất lượng
Accuracy, Precision, Recall, F1-score.

Hiệu năng
Latency, throughput, thời gian tải model.

Tài nguyên
RAM, VRAM, CPU/GPU utilization.

Layer / Operator
Thời gian thực thi từng layer và bottleneck.

Phương thức hoạt động tổng quát của hệ thống

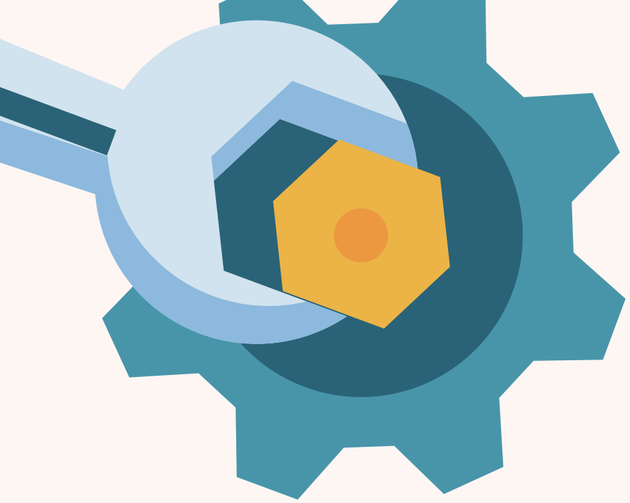


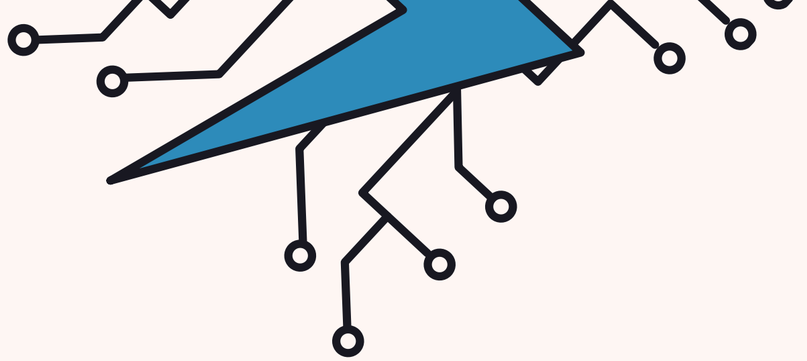
Nghiên cứu và phân tích tài liệu

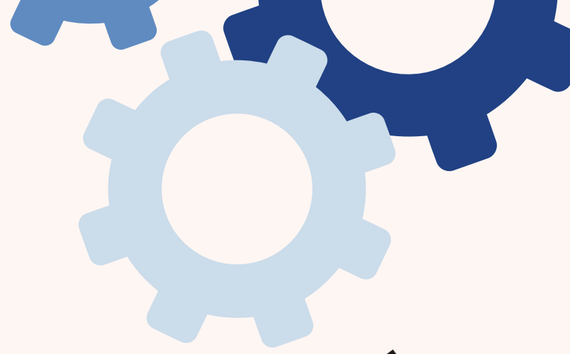
Nghiên cứu và phân tích tài liệu

Proof

MLModelScope

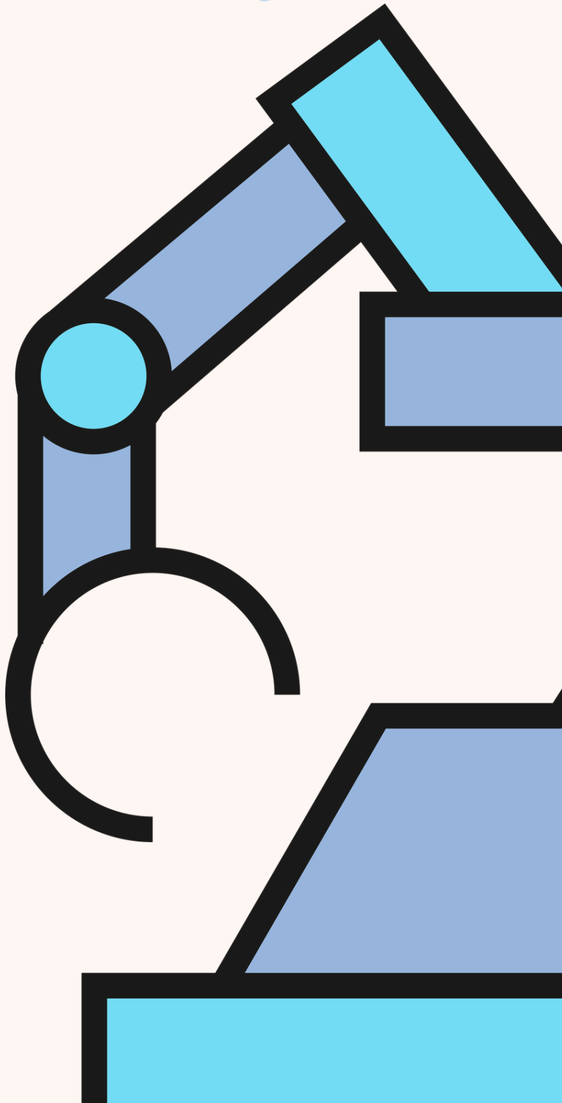


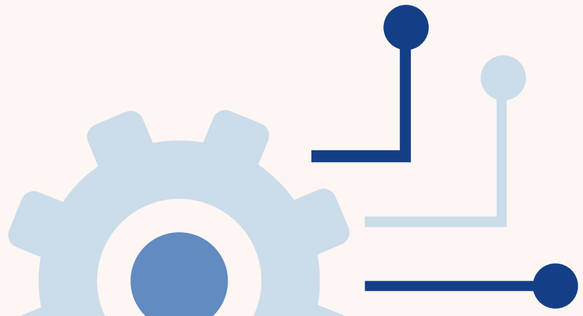




Proof

S. Wu, H. Yang, X. You, R. Gong, Y. Liu, Z. Luan, and D. Qian, “PRoof: A Comprehensive Hierarchical Profiling Framework for Deep Neural Networks with Roofline Analysis,” Proceedings of the 53rd International Conference on Parallel Processing (ICPP ’24), ACM, pp. 822–832, 2024.





PRoof là gì?

1 PRoof

- Nhận model định dạng ONNX.
- Profiling DNN inference theo góc nhìn phần cứng.
- Phân tích end-to-end và layer-wise.
- Liên kết model layer với backend layer và hardware metrics.
- Hỗ trợ tối ưu model và hardware.cho

2 Ý tưởng chính

Không chỉ hỏi: “model chạy mất bao lâu?”
PRoof trả lời thêm:

- Layer nào chậm?
- Chậm vì tính toán hay bộ nhớ?
- Backend layer đó tương ứng với phần nào trong model gốc?
- Cần tối ưu model hay phần cứng?

Hai kỹ thuật chính của PRoof

1. Layer Mapping

Mục tiêu: mapping ngược từ backend layer về original model layer.

Original: Conv → Add → ReLU
Runtime: Conv + Add + ReLU
Backend: Fused Layer

Khi fused layer chậm, ta biết nó đến từ layer nào trong model gốc.

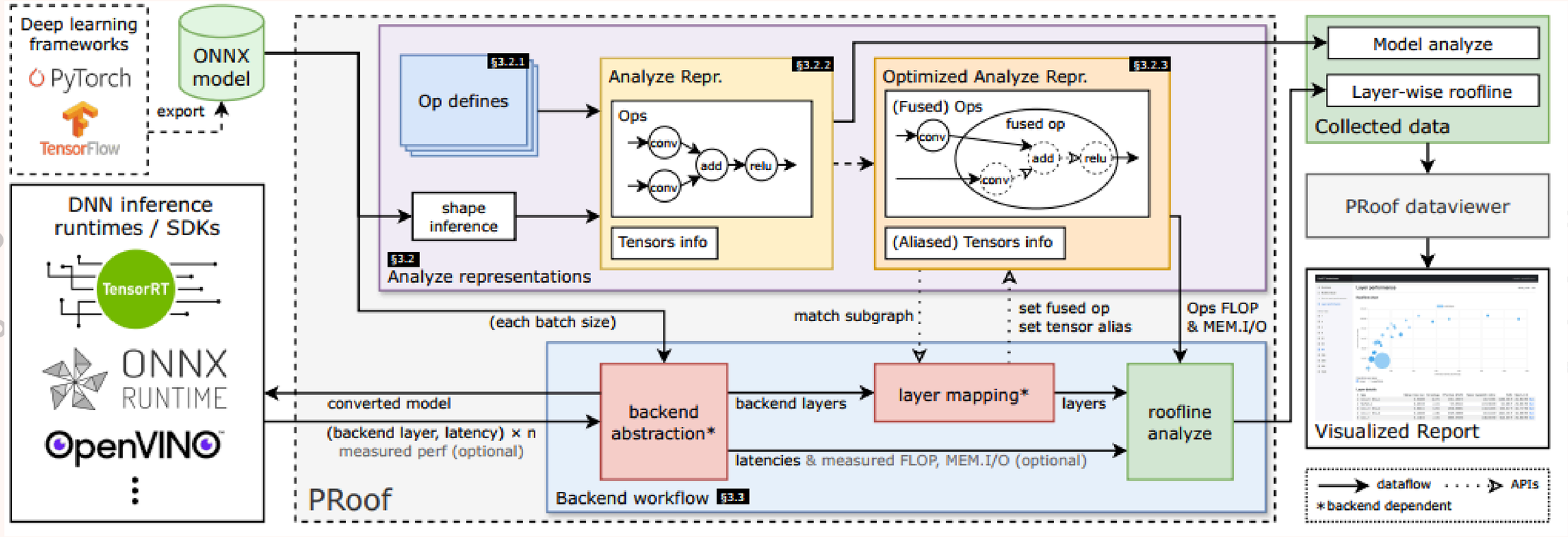
2. Analytical Performance Model

PRoof dự đoán:
- FLOP
- Memory Access

Dựa trên:
- Operator type
- Input/output shape
- Tensor size
- Model graph

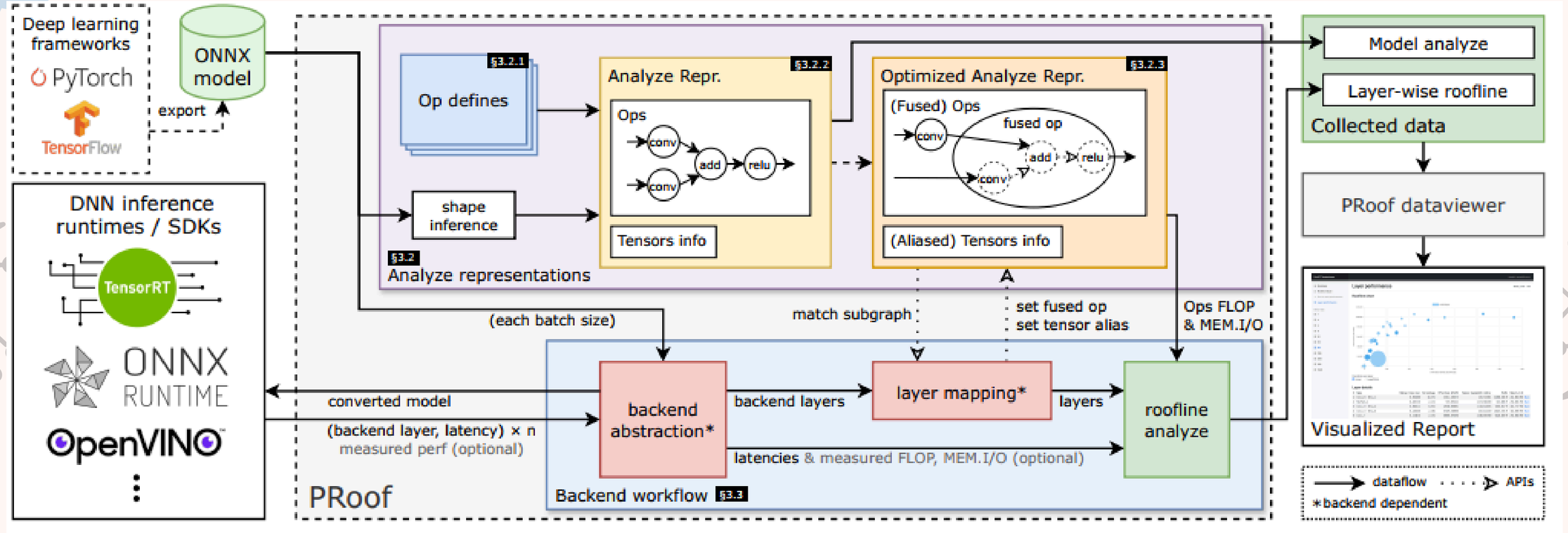
Nhờ đó giảm phụ thuộc vào hardware counter.

Kiến trúc của PRoof



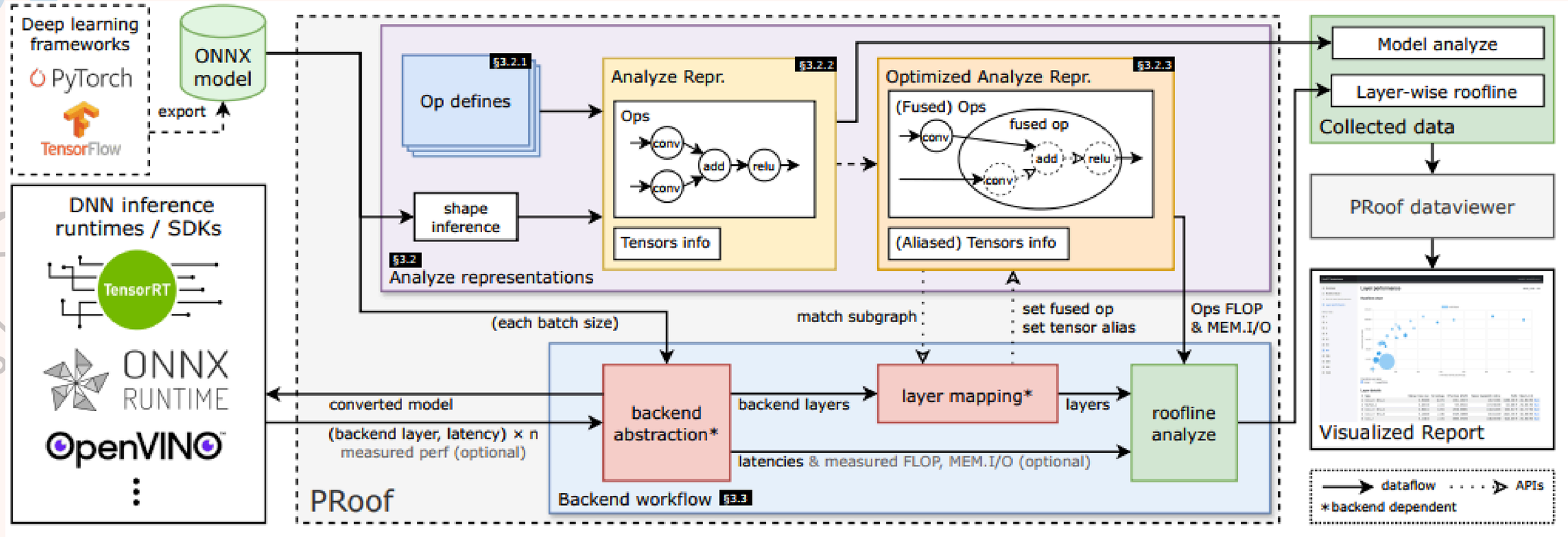
1. Export ONNX:Chuyển model từ PyTorch/TensorFlow sang ONNX để chuẩn hóa dầu vào cho PRoof.

Kiến trúc của PRoof



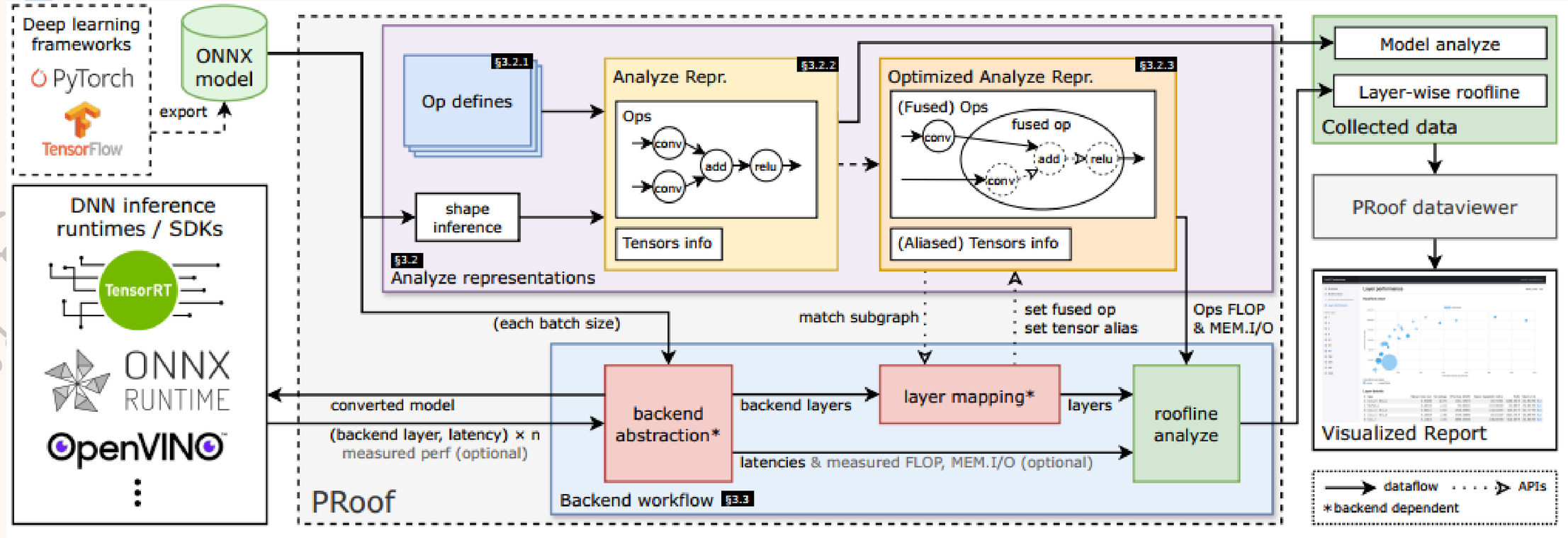
2. Model Analysis : Phân tích computational graph, xác định các operator, tensor và shape; dự đoán FLOP và memory access.

Kiến trúc của PRoof



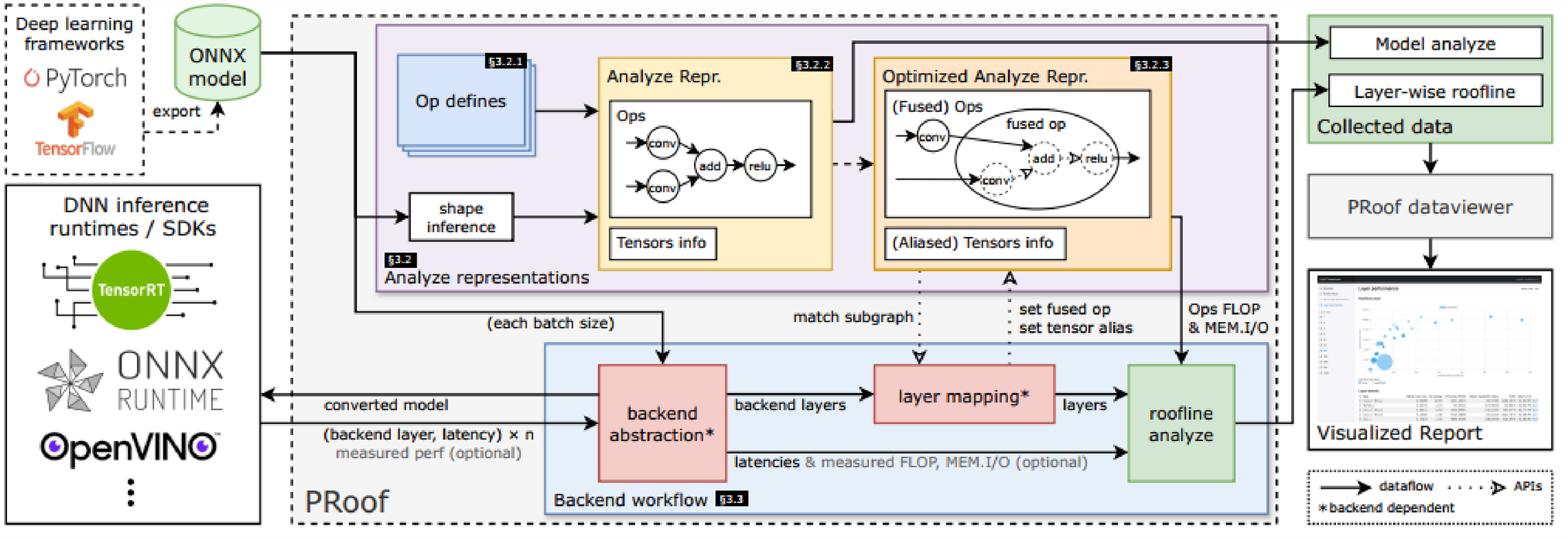
3. Backend Profiling : Chạy model trên TensorRT / ONNX Runtime / OpenVINO và thu thập latency của backend layer sau tối ưu.

Kiến trúc của PRoof



4.Layer Mapping & Roofline:Ánh xạ backend layer về layer gốc, kết hợp latency + FLOP + memory để xác định memory-bound hay compute-bound.

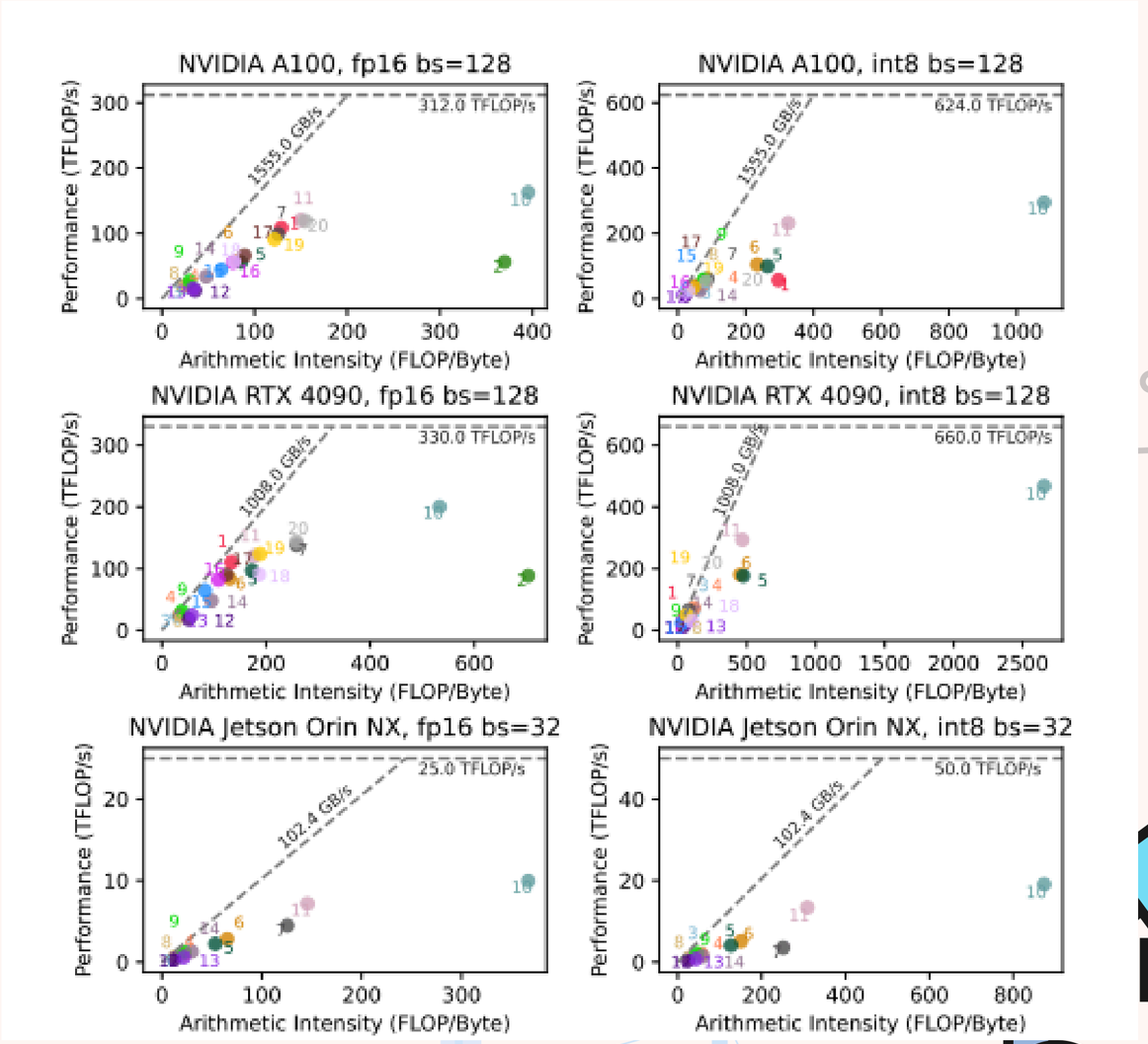
Kiến trúc của PRoof



5. DataViewer : Trực quan hóa latency, FLOP/s, memory bandwidth và Roofline chart để xác định bottleneck và hướng tối ưu.

Roofline Analysis

- Trục X: Arithmetic Intensity = FLOP / Byte
- Trục Y: Performance = FLOP/s
- Memory-bound: hiệu năng bị giới hạn bởi băng thông bộ nhớ
- Compute-bound: hiệu năng bị giới hạn bởi khả năng tính toán
- Mỗi điểm biểu diễn một model hoặc backend layer



Kết quả thực nghiệm của PRoof

Quy mô đánh giá

- 20 mô hình đại diện.
- 7 nền tảng phần cứng.
- Runtime: TensorRT, ONNX Runtime, OpenVINO.

Model gồm: ResNet, EfficientNet, MobileNet, ShuffleNet, ViT, Swin, BERT, Stable Diffusion.

Table 2: Hardware for evaluation.

Hardware	Scenarios	Runtime
NVIDIA A100 PCIE-40GB	Data center GPU	TensorRT 8.6.1
NVIDIA RTX 4090	Desktop GPU	TensorRT 8.6.1
Intel Xeon Gold 6330	Datacenter CPU	ONNX Runtime 1.15.0
NV. Jetson Xavier NX	Edge GPU	TensorRT 8.4.1
NV. Jetson Orin NX 16GB	Edge GPU	TensorRT 8.5.2
Raspberry Pi 4B	Edge CPU	ONNX Runtime 1.14.1
NPU 3720 (in Intel Ultra9 185H)	Mobile NPU	OpenVINO 2024.0.0

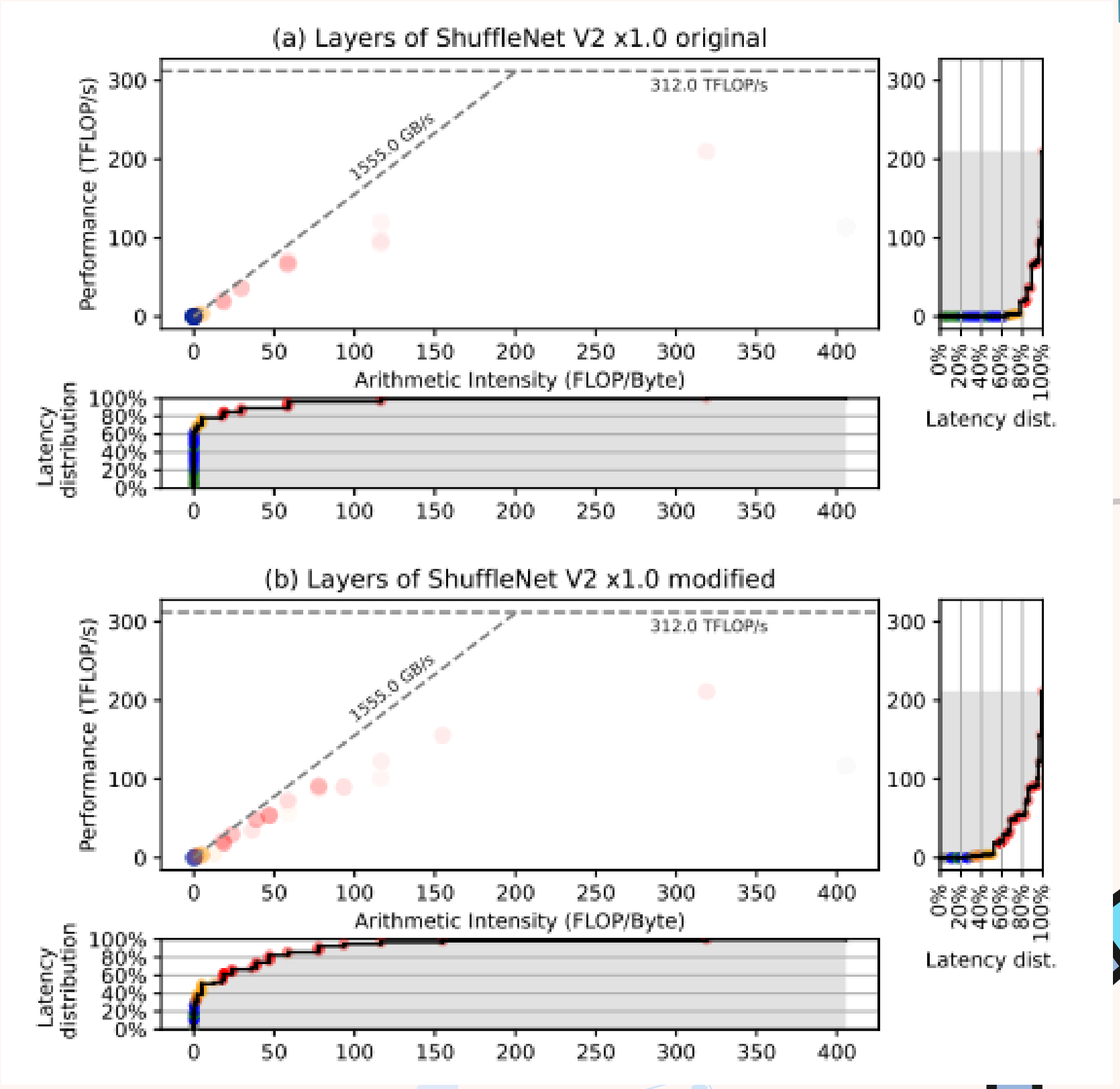
Table 3: Models for evaluation.

#	Model name	Type	ONNX Nodes	Params (M)	GFLOP*
1	DistilBERT base	Trans.	435	67.0	48.718
2	Stable Diffusion	Diffu.	5343	859.5	4747.726
3	EfficientNet B0	CNN	239	5.3	0.851
4	EfficientNet B4	CNN	476	19.3	3.209
5	EfficientNetV2-T	CNN	487	13.6	3.939
6	EfficientNetV2-S	CNN	504	23.9	6.030
7	MLP-Mixer (B_{16})	MLP	497	59.9	25.403
8	MobileNetV2 0.5	CNN	100	2.0	0.205
9	MobileNetV2 1.0	CNN	100	3.5	0.621
10	ResNet-34	CNN	89	21.8	7.338
11	ResNet-50	CNN	122	25.5	8.207
12	ShuffleNetV2 x0.5	CNN	584	1.4	0.084
13	ShuffleNetV2 x1.0	CNN	584	2.3	0.294
14	Shuf. v2 x1.0 mod	CNN	156	2.8	0.434
15	Swin tiny (P_4 W_7)	Trans.	1465	28.8	9.133
16	Swin small (P_4 W_7)	Trans.	2839	50.5	17.723
17	Swin base (P_4 W_7)	Trans.	2839	88.9	31.183
18	ViT tiny (P_4 W_7)	Trans.	786	5.7	2.558
19	ViT small (P_4 W_7)	Trans.	786	22.1	9.298
20	ViT base (P_4 W_7)	Trans.	786	86.6	35.329

Case study: Tối ưu ShuffleNetV2

Phát hiện bottleneck

- ShuffleNetV2 x1.0 đạt hiệu năng thấp so với peak của A100.
- PRoof cho thấy các lớp Transpose và Data Copy chiếm nhiều thời gian.
- Các thao tác này xuất hiện do Shuffle operation.
- Đây là các thao tác memory-intensive, làm tăng truy cập bộ nhớ.



Chú thích :

Depth-wise Conv | Conv khác / Point-wise | Transpose | Data Copy

Case study: Tối ưu ShuffleNetV2

Phương pháp tối ưu

- Loại bỏ Shuffle operation ở non-downsampling block.
- Tăng số channel ở các point-wise convolution để xử lý toàn bộ channel.
- thêm phép Add để cộng input ban đầu với đầu ra của nhánh convolution, tạo thành residual connection
- Giữ nguyên downsampling block.

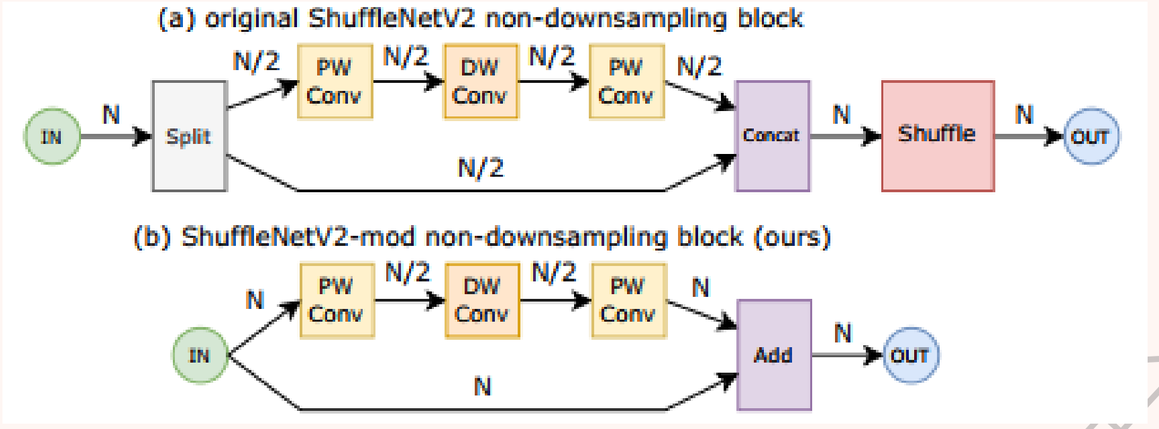
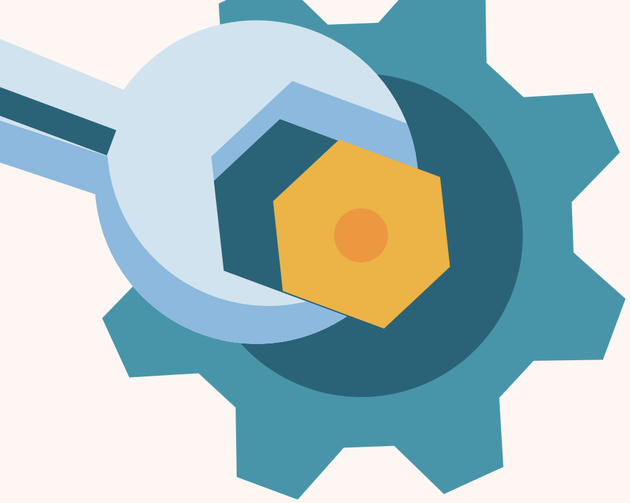
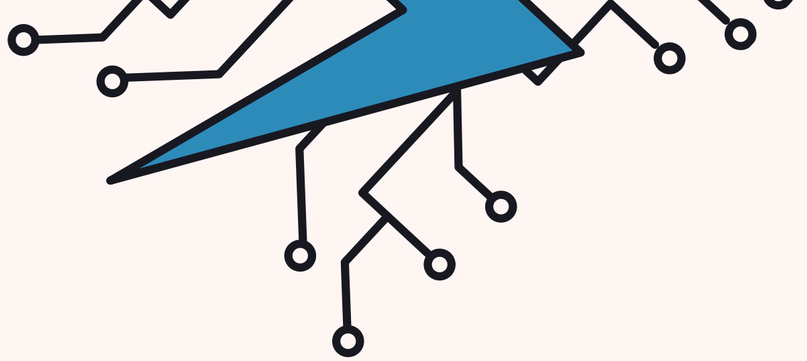
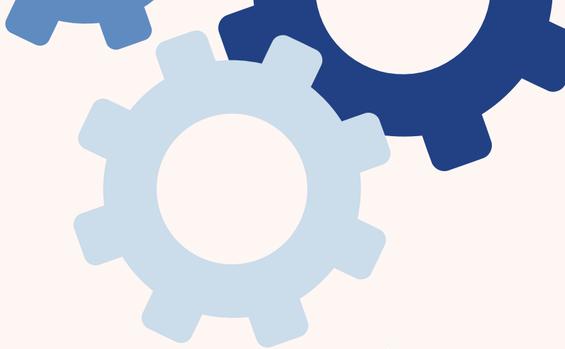


Table 5: Effectiveness of the modified *ShuffleNetV2 x1.0* model.

Model	Params (M)	ImageNet Top-1 Accuracy (%)	Batch size	GFLOP	Latency (ms)	Throughput (images/s)	Achieved GFLOP/s	Achieved memory bandwidth (GB/s)	Speedup
Original	2.271	68.9%	1	0.294	0.528	1894	556.759	34.026	-
			128	37.630	3.2479	39410	11585.843	530.486	-
			2048	602.079	49.543	41338	12152.612	555.062	-
Modified	2.804	70.1%	1	0.434	0.380	2632	1141.680	54.855	**1.39x**
			128	55.578	2.184	58608	25451.294	790.130	**1.49x**
			2048	889.255	30.126	67981	29518.047	895.042	**1.64x**

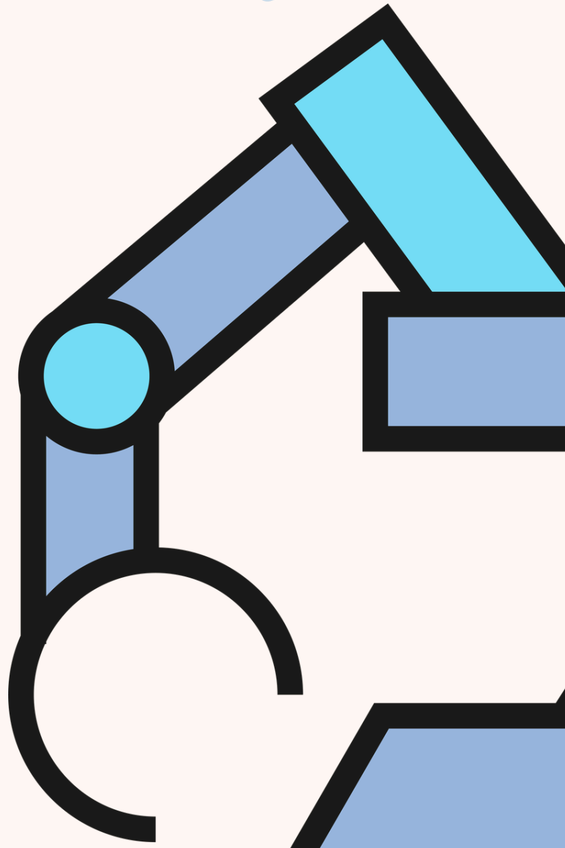


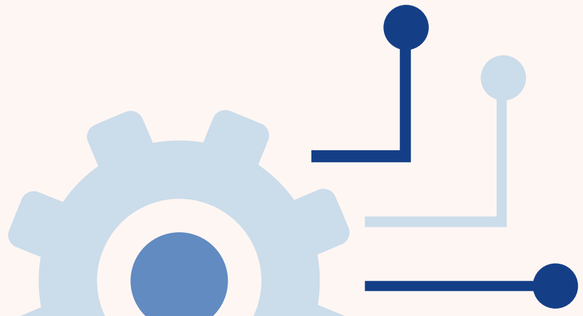




MLModelScope

A. Dakkak, C. Li, W.-m. Hwu, and J. Xiong, “MLModelScope: A Distributed Platform for Model Evaluation and Benchmarking at Scale,” arXiv preprint arXiv:2002.08295, 2020.





Mục tiêu và Yêu cầu của MLModelScope

1 Mục tiêu

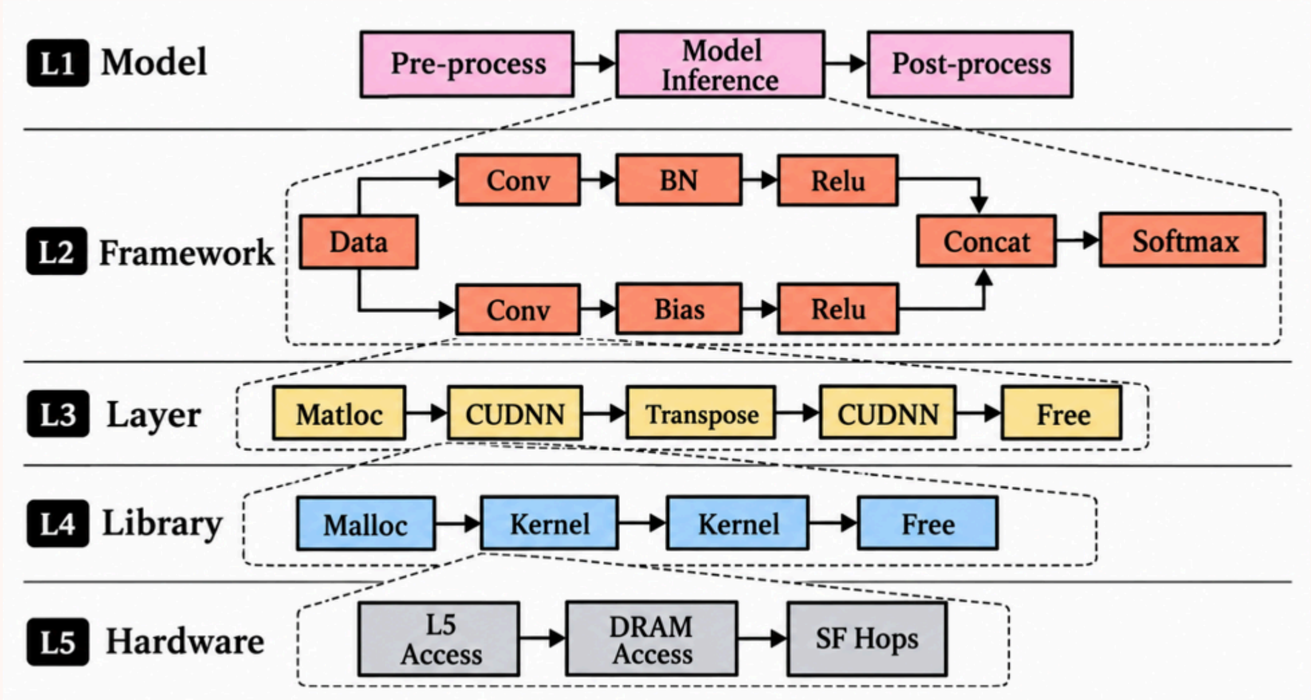
MLModelScope được xây dựng với mục tiêu chuẩn hóa và tự động hóa quá trình đánh giá, benchmark và thực thi các mô hình Machine Learning/Deep Learning trên nhiều môi trường phần cứng và phần mềm khác nhau

2 Yêu cầu

MLModelScope tập trung vào ba yêu cầu chính:

- **Repeatability**: Một thí nghiệm có thể được thực hiện lại trong cùng điều kiện.
- **Fairness**: khi so sánh hai mô hình hoặc hai hệ thống, các yếu tố không liên quan phải được giữ cố định.
- **Scalability**: có thể thực hiện nhiều thí nghiệm trên nhiều máy và nhiều loại phần cứng

Các mức phân tích của MLModelScope



L1 – Model lever: Bao gồm toàn bộ pipeline: Input → Pre-processing → Model inference → Post-processing → Output

Các mức phân tích của MLModelScope

L1 · Model

Input → pre-processing → model inference → post-processing → output. Đo trên dataset thật: latency mỗi batch và throughput ảnh thật, gồm chi phí Predictor chuyển input/output; không gồm preprocessing/postprocessing. Batch cuối có thể được padding. 35 batch đo · 10 lượt warmup không tính vào kết quả.

AVERAGE LATENCY	THROUGHPUT	BACKEND	DEVICE
107,636 ms	297,033 samples/s	ONNXRuntimeBackend	GPU

GFLOPS/bandwidth bổ sung là ước tính từ tổng FLOPs/byte của các module đã ghi nhận, chia latency inference hiện tại. Không phải bộ đếm phần cứng; không bao gồm phép toán ngoài module chưa được thu thập. Memory là lưu lượng logic, không phải RAM/VRAM cấp phát.

EFFECTIVE GFLOPS	MEMORY BANDWIDTH	MIN LATENCY	MAX LATENCY
1.080,53	15,05 GB/s	30,892 ms	314,182 ms

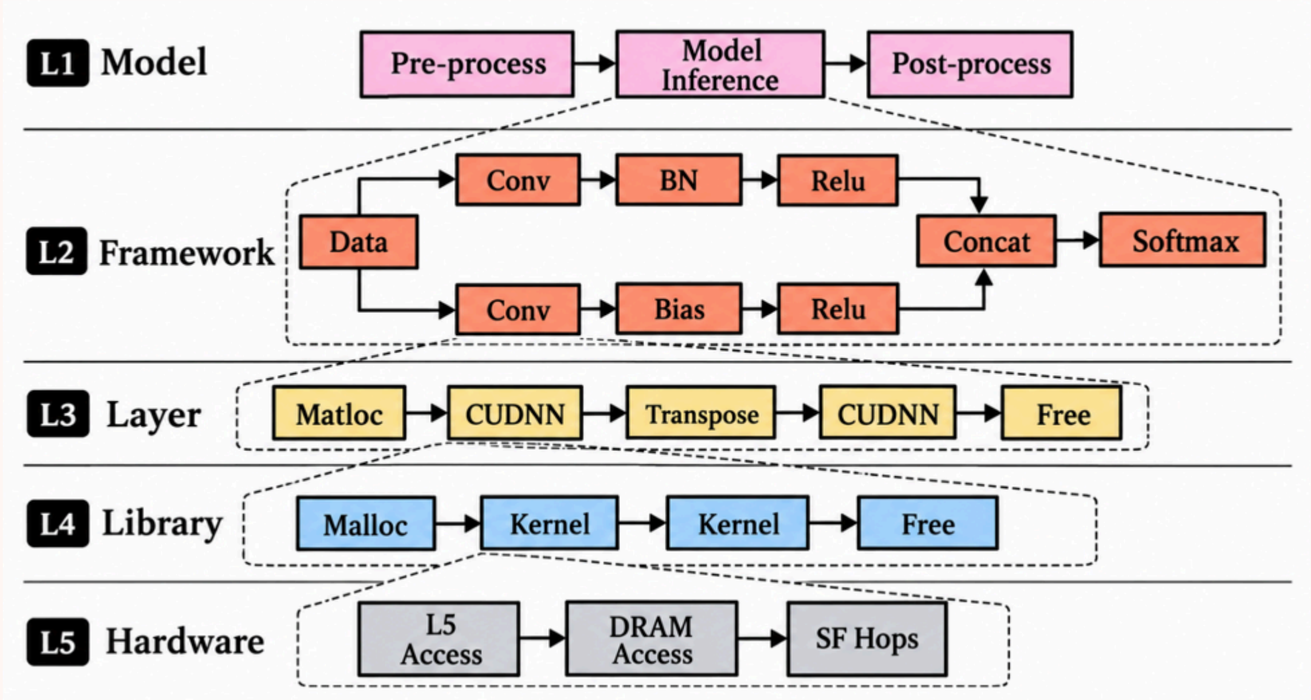
Dataset pipeline & evaluation

Ảnh thật → Preprocess → Inference → Postprocess → Evaluation. Accuracy chỉ đếm mỗi ảnh thật một lần; ảnh padding không được tính vào số mẫu.
Pipeline đo từ đọc ảnh/preprocess đến kết thúc postprocess, gồm truyền dữ liệu và chi phí gọi Python. Warmup và evaluation đo riêng, không cộng vào pipeline. Các thời gian dưới đây là tổng trên dataset; latency là trung bình mỗi batch.

PIPELINE LATENCY / BATCH	PIPELINE THROUGHPUT
1.040,031 ms	30,741 samples/s

Metrics tỷ lệ 0–1 dùng thang 0–1. Các chỉ số hiệu năng có thang riêng, bắt đầu từ 0; các run dùng chung thang trong cùng chỉ số. Số hiển thị giữ nguyên đơn vị metric trong report.

Các mức phân tích của MLModelScope

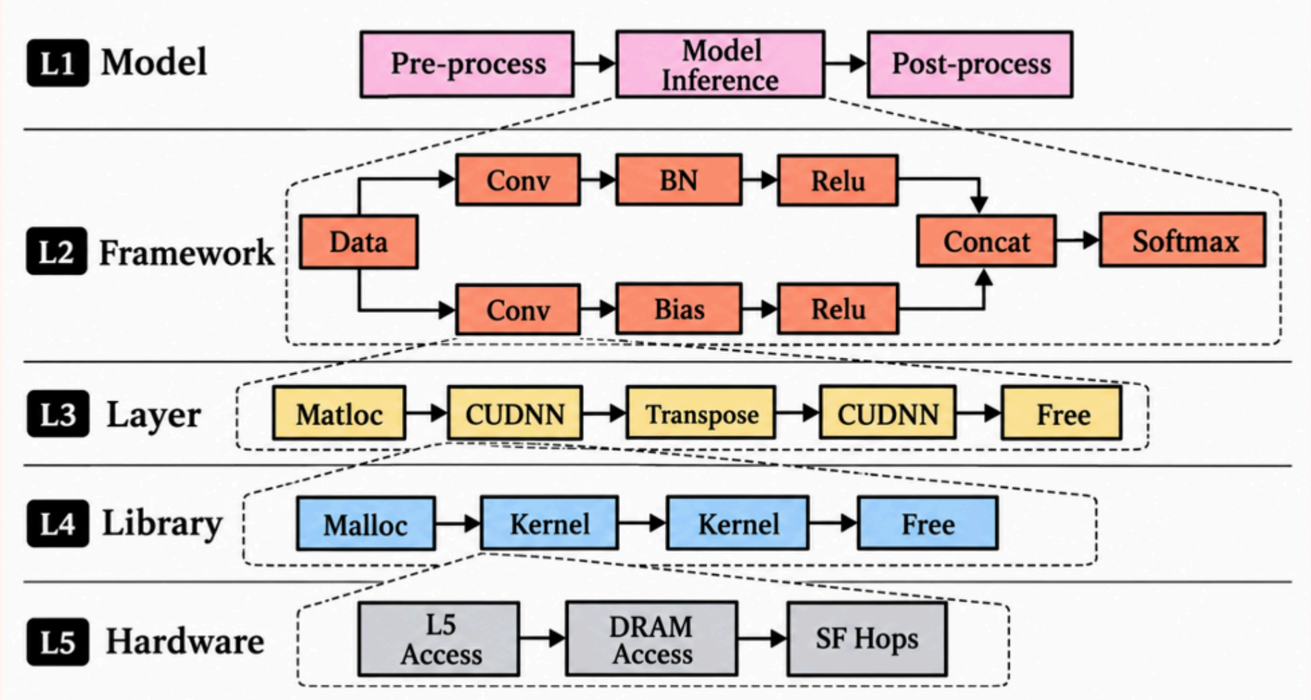


L2 – Framework level: Phân tích computational graph của mô hình ở mức framework, bao gồm các toán tử như Conv, Softmax..

Các mức phân tích của MLModelScope

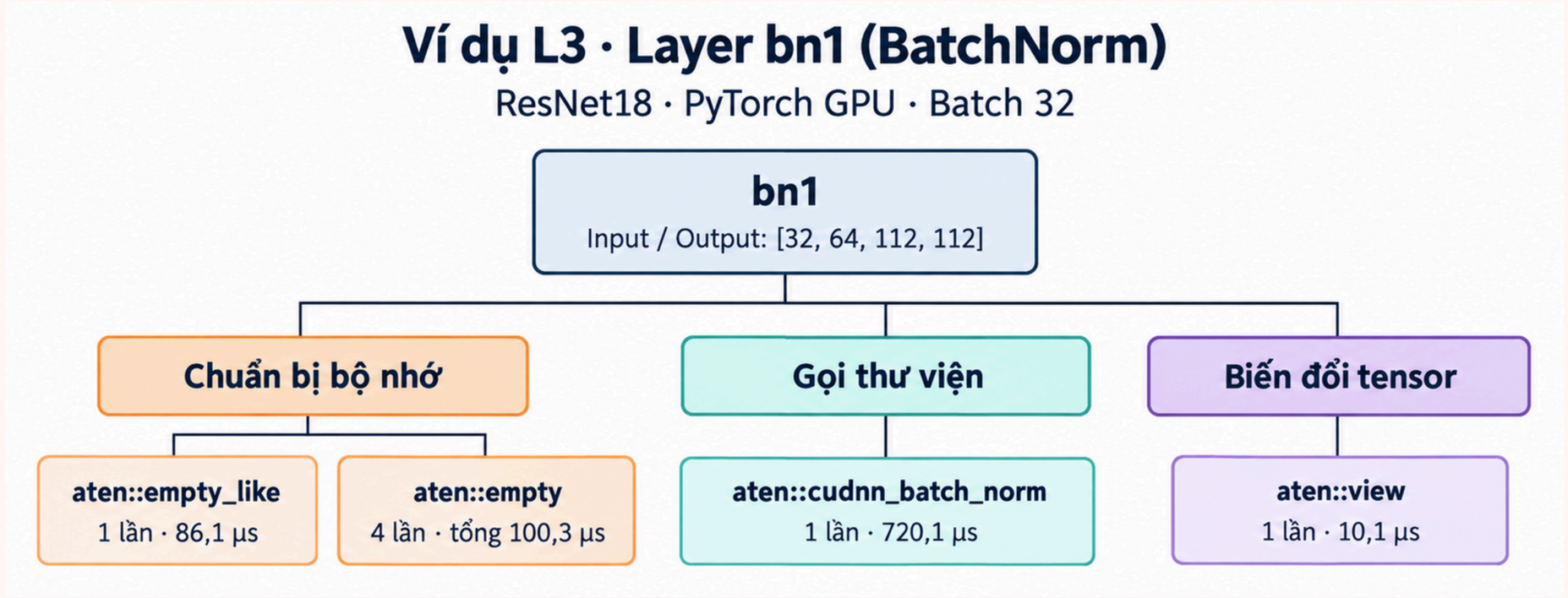
#	Layer	Type	Input shape	Output shape	Params	Latency (ms)	%	Effective GFLOPS	Bandwidth (GB/s)	FLOPs	Memory
1	layer4.1.conv2	Conv2d	[32,512,7,7]	[32,512,7,7]	2.359.296	7,242	9,95%	1.021,687	2,19	7.398.752.256	15.13 MB
2	layer2.0.downsample.0	Conv2d	[32,64,56,56]	[32,128,28,28]	8.192	6,278	8,63%	65,472	6,143	411.041.792	36.78 MB
3	layer2.1.conv2	Conv2d	[32,128,28,28]	[32,128,28,28]	147.456	6,234	8,57%	1.186,858	4,216	7.398.752.256	25.06 MB
4	layer1.1.conv2	Conv2d	[32,64,56,56]	[32,64,56,56]	36.864	5,92	8,13%	1.249,81	8,704	7.398.752.256	49.14 MB
5	layer2.1.conv1	Conv2d	[32,128,28,28]	[32,128,28,28]	147.456	5,406	7,43%	1.368,543	4,861	7.398.752.256	25.06 MB

Các mức phân tích của MLModelScope

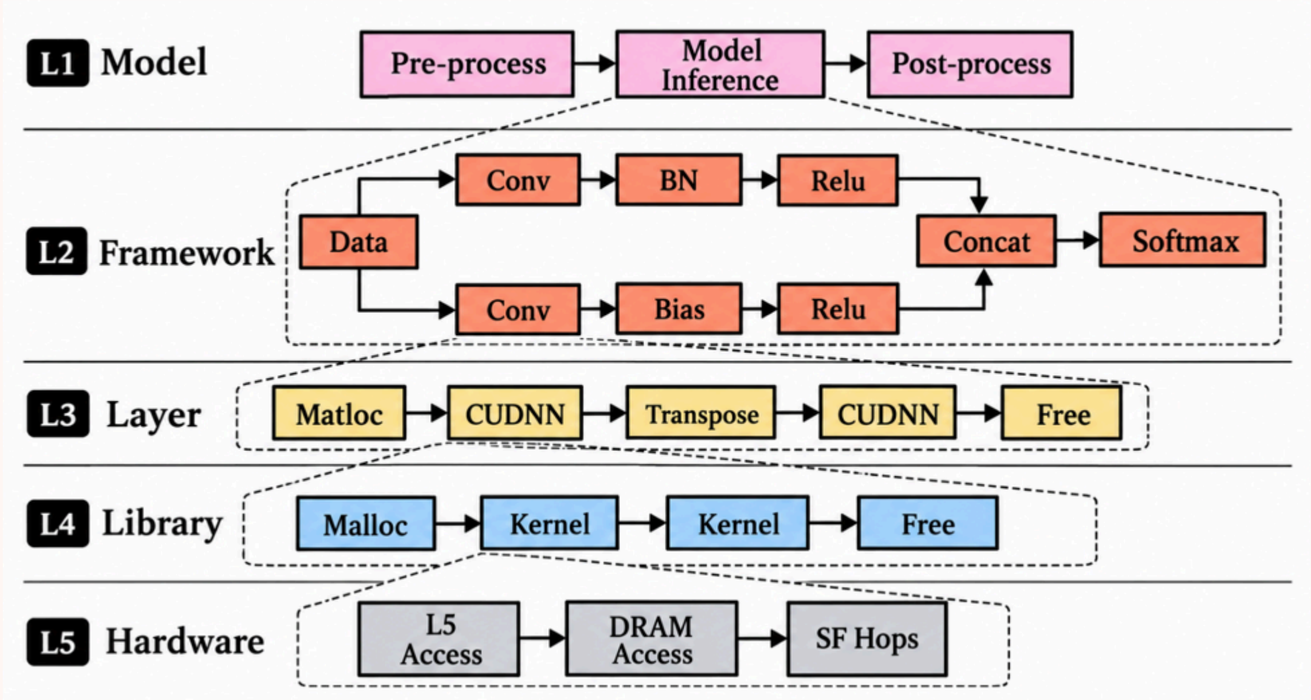


L3 – Layer level: Phân tích cách từng layer/toán tử được thực thi ở mức thấp, gồm cấp phát bộ nhớ, gọi thư viện và các thao tác phụ...

Các mức phân tích của MLModelScope

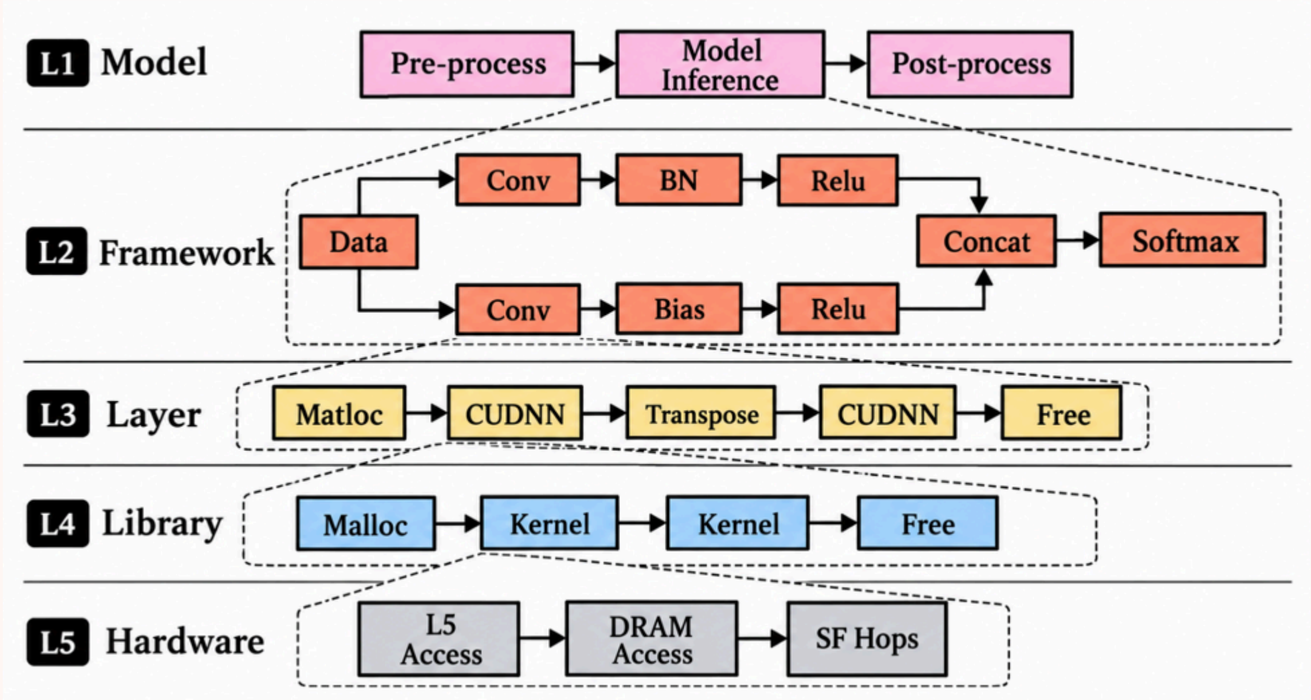


Các mức phân tích của MLModelScope



L4 – Library level: Phân tích các thư viện và kernel thực thi bên dưới như CUDA, cuDNN,... để xác định hàm/kernel nào chiếm nhiều thời gian và trở thành nút thắt hiệu năng.

Các mức phân tích của MLModelScope



L5 – Hardware level: Theo dõi hoạt động trên phần cứng như cache, DRAM/VRAM và các phép toán tính toán để xác định chương trình đang bị giới hạn bởi năng lực tính toán hay băng thông bộ nhớ.

Kiến trúc của MLModelScope

MLModelScope bao gồm 2 thành phần chính:

- **Model Evaluation Manifest**: Là file cấu hình đặc tả đầy đủ thông tin cần thiết để chạy model
- **MLModelScope Runtime**: là thành phần chịu trách nhiệm điều phối toàn bộ quá trình đánh giá và benchmark model dựa trên thông tin trong model manifest và các ràng buộc do người dùng cung cấp như tên model, version, framework và hardware.

Model Evaluation Manifest

```yaml
name: Inception-v3 # model name
version: 1.0.0 # semantic version of model
task: classification # model modality
license: MIT # model license
description: ...
framework: # framework information
  name: TensorFlow
  version: ^1.x # framework version constraint
container: # containers used for architecture
  arm64: mlms/tensorflow:1-13-0_arm64-cpu
  amd64:
    cpu: mlms/tensorflow:1-13-0_amd64-cpu
    gpu: mlms/tensorflow:1-13-0_amd64-gpu
  ppc64le:
    cpu: mlms/tensorflow:1-13-0_ppc64le-cpu
    gpu: mlms/tensorflow:1-13-0_ppc64le-gpu
envvars:
  TF_ENABLE_WINOGRAD_NONFUSED: 0
inputs: # model inputs
  - type: image # first input modality
    layer_name: data
    element_type: float32
preproccessing: |
  def pre_processing(env, inputs):
      # e.g. import opencv as cv
      return preproc_inputs
outputs: # model outputs
  - type: probability # output modality
    layer_name: prob
    element_type: float32
post-processing: |
  def post_processing(env, inputs):
      # e.g. os.exec("Rscript ~/postproc.r")
      return postproc_inputs
source: # model source
  graph_path: https://.../inception_v3.pb
training_data: # dataset used for training
  name: ILSVRC 2012
  version: 1.0.0
```

Nhóm thông tin cơ bản của model:

- name: tên model
- version: phiên bản model
- task: loại bài toán
- license
- description

Model Evaluation Manifest

```yaml
name: Inception-v3 # model name
version: 1.0.0 # semantic version of model
task: classification # model modality
license: MIT # model license
description: ...
framework: # framework information
  name: TensorFlow
  version: ^1.x # framework version constraint
container: # containers used for architecture
  arm64: mlms/tensorflow:1-13-0_arm64-cpu
  amd64:
    cpu: mlms/tensorflow:1-13-0_amd64-cpu
    gpu: mlms/tensorflow:1-13-0_amd64-gpu
  ppc64le:
    cpu: mlms/tensorflow:1-13-0_ppc64le-cpu
    gpu: mlms/tensorflow:1-13-0_ppc64le-gpu
envvars:
  TF_ENABLE_WINOGRAD_NONFUSED: 0
inputs: # model inputs
  - type: image # first input modality
    layer_name: data
    element_type: float32
preproccessing: |
  def pre_processing(env, inputs):
      # e.g. import opencv as cv
      return preproc_inputs
outputs: # model outputs
  - type: probability # output modality
    layer_name: prob
    element_type: float32
post-processing: |
  def post_processing(env, inputs):
      # e.g. os.exec("Rscript ~/postproc.r")
      return postproc_inputs
source: # model source
  graph_path: https://.../inception_v3.pb
training_data: # dataset used for training
  name: ILSVRC 2012
  version: 1.0.0
```

Nhóm Software stack: Mô tả phần mềm cần để chạy model gồm:

- framework nào;
- version framework;
- Docker container nào;
- container tương ứng với kiến trúc máy;
- các environment variable cần thiết.

Model Evaluation Manifest

```yaml
name: Inception-v3 # model name
version: 1.0.0 # semantic version of model
task: classification # model modality
license: MIT # model license
description: ...
framework: # framework information
  name: TensorFlow
  version: ^1.x # framework version constraint
container: # containers used for architecture
  arm64: mlms/tensorflow:1-13-0_arm64-cpu
  amd64:
    cpu: mlms/tensorflow:1-13-0_amd64-cpu
    gpu: mlms/tensorflow:1-13-0_amd64-gpu
  ppc64le:
    cpu: mlms/tensorflow:1-13-0_ppc64le-cpu
    gpu: mlms/tensorflow:1-13-0_ppc64le-gpu
envvars:
  TF_ENABLE_WINOGRAD_NONFUSED: 0
inputs: # model inputs
  - type: image # first input modality
    layer_name: data
    element_type: float32
preproccessing: |
  def pre_processing(env, inputs):
      # e.g. import opencv as cv
      return preproc_inputs
outputs: # model outputs
  - type: probability # output modality
    layer_name: prob
    element_type: float32
post-processing: |
  def post_processing(env, inputs):
      # e.g. os.exec("Rscript ~/postproc.r")
      return postproc_inputs
source: # model source
  graph_path: https://.../inception_v3.pb
training_data: # dataset used for training
  name: ILSVRC 2012
  version: 1.0.0
```

Nhóm Model Input + Pre – processing:

- Input ghi kiểu dữ liệu đầu vào ví dụ type: Image, Layer_name, element type...
- Pre – processing có 2 cách biểu diện: Python function và operation built in

```yaml
type: image  # input modality
layer_name: data
element_type: float32
steps: # pre-processing steps
  decode:
    element_type: int8
    data_layout: NHWC
    color_layout: RGB
  crop:
    method: center
    percentage: 87.5
  resize:
    dimensions: [3, 299, 299]
    method: bilinear
    keep_aspect_ratio: true
  mean: [127.5, 127.5, 127.5]
  rescale: 127.5
```

Model Evaluation Manifest

```yaml
name: Inception-v3 # model name
version: 1.0.0 # semantic version of model
task: classification # model modality
license: MIT # model license
description: ...
framework: # framework information
  name: TensorFlow
  version: ^1.x # framework version constraint
container: # containers used for architecture
  arm64: mlms/tensorflow:1-13-0_arm64-cpu
  amd64:
    cpu: mlms/tensorflow:1-13-0_amd64-cpu
    gpu: mlms/tensorflow:1-13-0_amd64-gpu
  ppc64le:
    cpu: mlms/tensorflow:1-13-0_ppc64le-cpu
    gpu: mlms/tensorflow:1-13-0_ppc64le-gpu
envvars:
  TF_ENABLE_WINOGRAD_NONFUSED: 0
inputs: # model inputs
  - type: image # first input modality
    layer_name: data
    element_type: float32
preproccessing: |
  def pre_processing(env, inputs):
      # e.g. import opencv as cv
      return preproc_inputs
outputs: # model outputs
  - type: probability # output modality
    layer_name: prob
    element_type: float32
post-processing: |
  def post_processing(env, inputs):
      # e.g. os.exec("Rscript ~/postproc.r")
      return postproc_inputs
source: # model source
  graph_path: https://.../inception_v3.pb
training_data: # dataset used for training
  name: ILSVRC 2012
  version: 1.0.0
```

Nhóm Model Output + Post – processing:

- Tương tự như Input thì Output cũng có các mô tả ví dụ: type: probability...
- Post – processing chịu trách nhiệm đưa output thô thành dạng có nghĩa ví dụ: class= dog

Model Evaluation Manifest

```yaml
name: Inception-v3 # model name
version: 1.0.0 # semantic version of model
task: classification # model modality
license: MIT # model license
description: ...
framework: # framework information
  name: TensorFlow
  version: ^1.x # framework version constraint
container: # containers used for architecture
  arm64: mlms/tensorflow:1-13-0_arm64-cpu
  amd64:
    cpu: mlms/tensorflow:1-13-0_amd64-cpu
    gpu: mlms/tensorflow:1-13-0_amd64-gpu
  ppc64le:
    cpu: mlms/tensorflow:1-13-0_ppc64le-cpu
    gpu: mlms/tensorflow:1-13-0_ppc64le-gpu
envvars:
  TF_ENABLE_WINOGRAD_NONFUSED: 0
inputs: # model inputs
  - type: image # first input modality
    layer_name: data
    element_type: float32
preproccessing: |
  def pre_processing(env, inputs):
      # e.g. import opencv as cv
      return preproc_inputs
outputs: # model outputs
  - type: probability # output modality
    layer_name: prob
    element_type: float32
post-processing: |
  def post_processing(env, inputs):
      # e.g. os.exec("Rscript ~/postproc.r")
      return postproc_inputs
source: # model source
  graph_path: https://.../inception_v3.pb
training_data: # dataset used for training
  name: ILSVRC 2012
  version: 1.0.0
```

Nhóm Model Source: Cho biết đường dẫn model thực sự năm ở đâu

- Model graph: Cho biết kiến trúc mạng ví dụ: graph path:...
- Model weight: Các tham số, trọng số đã học

MLModelScope Runtime

MLModelScope Runtime

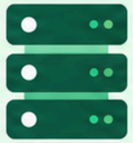
Gồm 5 thành phần quan trọng:

1 Predictor API

- `ModelLoad()`
- `Predict()`
- `ModelUnload()`

- Giao diện thống nhất cho các framework.
- Bao bọc API inference (C++) để đánh giá đồng nhất và tái sử dụng mã.

2 Agent



- Chạy trên một máy cụ thể.
- Xử lý manifest, tải model/dataset.
- Tiền xử lý, gọi Predictor API, hậu xử lý.
- Thu thập profiling và xuất kết quả.

3 Registry

- Lưu thông tin và trạng thái các agent.
- Giúp tìm agent phù hợp với yêu cầu phần cứng và phần mềm.

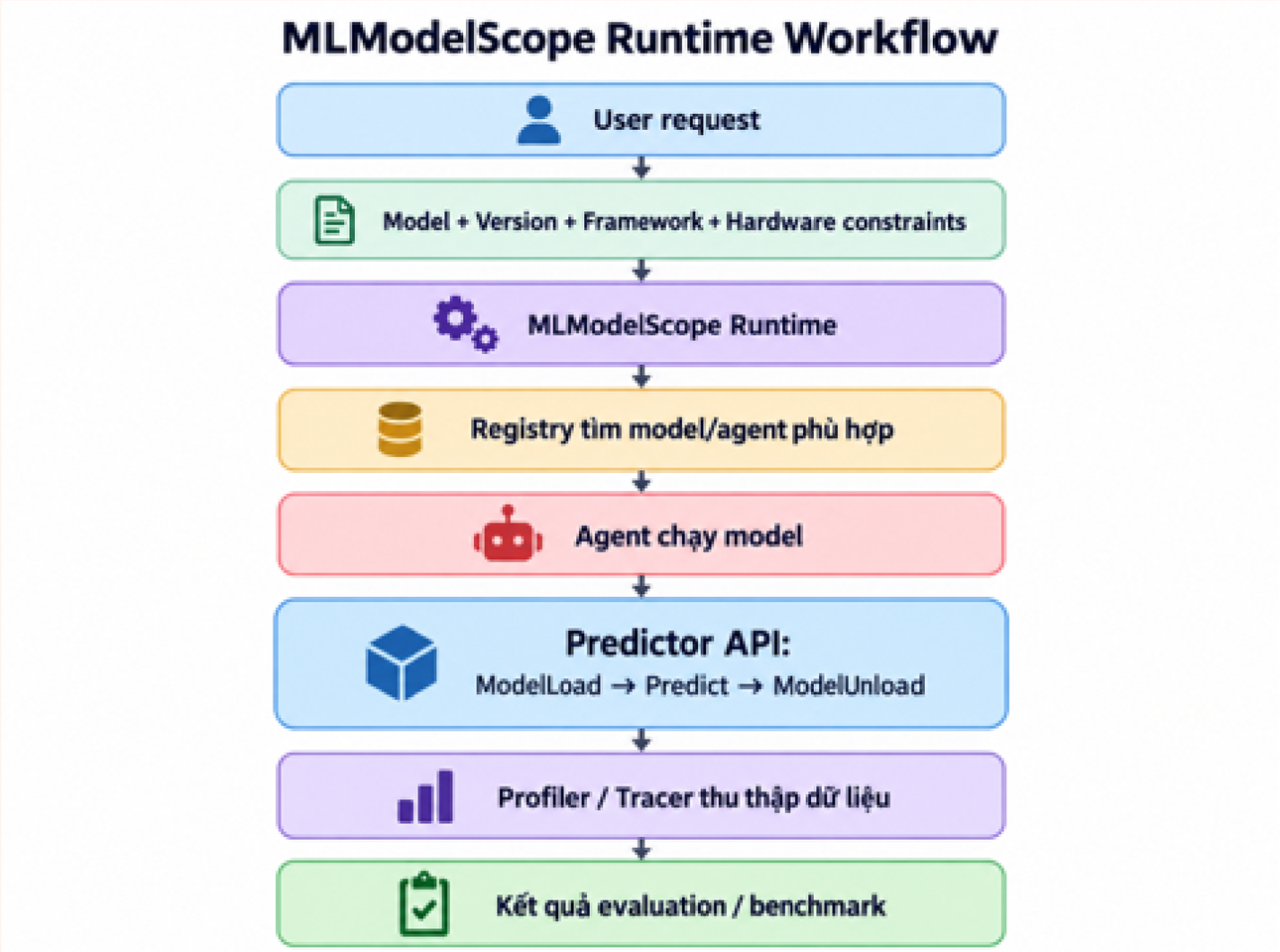
4 Orchestration Layer

- Điều phối quá trình thực thi.
- Chọn và route request tới agent phù hợp.
- Quản lý nhiều agent và hỗ trợ chạy song song.

5 Profiler và Tracer

- Thu thập thông tin hiệu năng ở mức model, framework và system.
- Hỗ trợ phân tích bottleneck.

MLModelScope Runtime



So sánh với proof

PRoof và MLModelScope

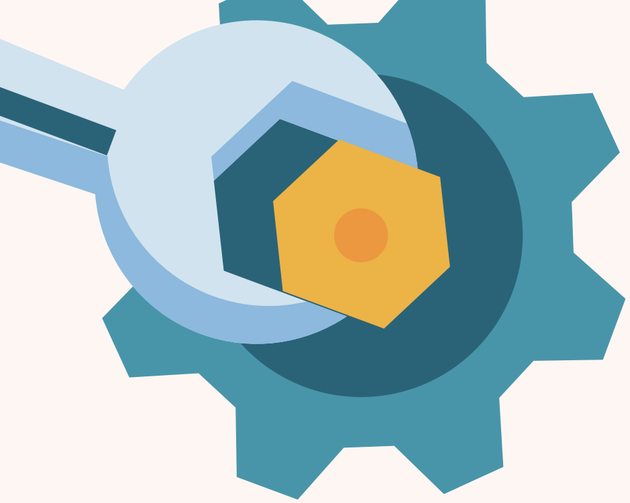
So sánh phạm vi phân tích và workload

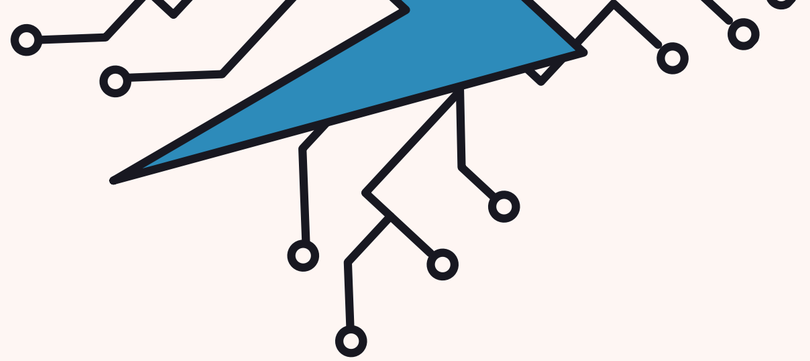
Tiêu chí	PRoof — luồng đang sử dụng	MLModelScope — kiến trúc
Mức phân tích	Chủ yếu L1: toàn model và L2: từng layer	Quan sát nhiều tầng: model → framework → layer → library → hardware
Model / framework	Nhận model ONNX; chạy qua backend như ONNX Runtime, TensorRT, OpenVINO	Agent cho nhiều framework, ví dụ PyTorch, TensorFlow, ONNX Runtime
Dữ liệu benchmark	Input tạo theo shape; chưa tích hợp pipeline dataset ảnh thật	Hỗ trợ workload/dataset và các bước xử lý dữ liệu
Trọng tâm	Latency • FLOPs • Memory • Roofline	Đánh giá pipeline và liên kết trace qua các tầng

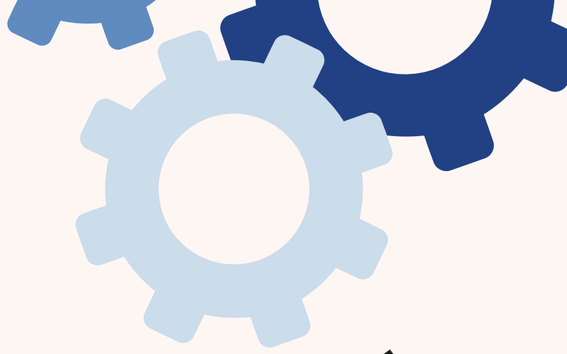
Lưu ý: PRoof không bị giới hạn tuyệt đối ở L1–L2; chế độ TensorRT + Nsight Compute có thể thu thêm dữ liệu kernel/phần cứng.
Mức L1–L5 là cách đối chiếu theo sơ đồ MLModelScope; độ sâu thu thập phụ thuộc backend và collector.

Nguồn: PRoof-framework/PRoof • MLModelScope (arXiv:1811.09737v1)

Demo sản phẩm







Thank You